Convolutional Networks: Locality and Translation Equivariance as Architecture

This is the second post in a series on neural-network inductive biases. The first post (the foundations post (`01-foundations.md`)) built logistic regression, softmax regression, and the multi-layer perceptron in pure Python, and named two distinct axes at which inductive bias enters every supervised model: the output head (link function and matching loss) and the architecture (how the network computes features). This post follows the architecture axis on its first non-trivial step: convolutional networks.

Setting up

The foundations post's MLP classifies 8x8 UCI digits at about 96% test accuracy. That looks reasonable until you ask why. The model treats each of the 64 pixels as an independent feature. Pixel $(3, 4)$ and pixel $(3, 5)$, which are neighbors in the image, are no more related in the MLP than pixel $(3, 4)$ and pixel $(7, 0)$, which are far apart. The MLP has no concept of "adjacency." It learns the relationships between pixels from data, like it would learn relationships between any other tabular features.

For a human, those features are not interchangeable. Permute the 64 pixels with a fixed random shuffle and the resulting "image" is unreadable. The MLP, by contrast, is unaffected: train it on the permuted data and it hits the same accuracy. That equivalence is the diagnostic. The MLP's prior is "any coordinate may interact with any other on equal footing." For images this is too weak. We are throwing away information about *how the data is arranged in space*.

A convolutional network is what we build when we commit to a stronger prior. The commitment has two parts. **Locality**: a unit looks at a small neighborhood, not the whole input. **Translation equivariance**: the same look is applied at every position. Both are baked into the architecture, not learned from data. That is the entire move, and the rest of the post is the unpacking.

1. One unit, reused everywhere

A `Linear` layer is one or more weight vectors $\mathbf{w}$ paired with biases b, computing a logit $z = \mathbf{w} \cdot \mathbf{x} + b$. A convolutional unit is the same thing, applied at every position of a 2-D input. We restrict $\mathbf{w}$ to a small **kernel** of size $k \times k$ and slide it across the image:

$$z_{r,c} = \sum_{i=0}^{k-1} \sum_{j=0}^{k-1} W_{i,j}\, x_{r+i,\, c+j} + b.$$

The kernel W is shared across all output positions (r, c). There is *one* W, not one per position. That is the entire definition of a convolution. Everything else is bookkeeping.

The bookkeeping covers three things:

- **Borders.** When the kernel runs off the edge of the input, we either drop those positions (`padding=0`, the choice this post uses) or pad the input with zeros to preserve spatial size.
- **Stride.** Sliding by one pixel between applications is the default (`stride=1`); larger strides downsample the output.

- **Channels.** A real image has multiple input channels (RGB has three). A convolutional layer has multiple output channels too, one per independent kernel run in parallel. With C_{in} input channels and C_{out} output channels, a kernel has shape (C_{in}, k, k) (it dots through every input channel at each spatial position) and a layer has C_{out} such kernels stacked.

For 8x8 single-channel digits with $k = 3$, padding 0, stride 1, and 4 output channels, the layer's output is shape $(4, 6, 6)$. Four feature maps, each 6x6 because the kernel cannot fit at the last two positions in each dimension.

2. Weight sharing is the translation prior

The reuse of W at every position is what makes the layer **translation equivariant**. Stated cleanly: if x' is x shifted by $(\Delta r, \Delta c)$, then the layer's output on x' is its output on x shifted by the same $(\Delta r, \Delta c)$ (modulo what falls off the borders).

This is not a property the network *learns*. It is a property of the *operator*: true at initialization, true after training, true for every kernel. Why? Because the same W is applied at every position. Shift the input by one and every windowed dot product is computed at a position that is also shifted by one. The output simply slides.

The MLP, by contrast, has no reason to be equivariant. Each output unit has its own $\mathbf{w}$, tied to specific positions in the input vector. Translating the input scrambles which weights see which pixels.

So weight sharing *is* translation equivariance. They are the same fact, seen from the parameter side and from the function-symmetry side.

3. The parameter count tells the story

The compactness of weight sharing shows up immediately in the parameter count.

The foundations post's MLP for 8x8 digits has shape $64 \to 32 \to 10$. Its parameter count is

$$64 \cdot 32 + 32 + 32 \cdot 10 + 10 = 2410.$$

A small CNN for the same task uses one conv layer of four 3x3 kernels, followed by a fully connected head:

```
Conv2D(in_channels=1, out_channels=4, kernel_size=3)
ReLU()
Linear(144, 10)
```

The conv layer has $4 \cdot (1 \cdot 9) + 4 = 40$ parameters. After `padding=0` and `stride=1`, the output is $4 \times 6 \times 6 = 144$ units, so the head is `Linear(144, 10)` with $144 \cdot 10 + 10 = 1450$ parameters. The total is

$$40 + 1450 = 1490,$$

about 38% smaller than the MLP. Most of the model's spatial capacity sits in the *40 parameters of the conv kernel*. The head is a thin reader on top.

To make the comparison sharper, ask: what would it cost to replace the conv layer with a `Linear` layer producing the same 144-dimensional output? That layer would be `Linear(64, 144)`, costing $64 \cdot 144 + 144 = 9360$ parameters. The conv layer accomplishes the same input-to-output shape transformation with 40 parameters instead of 9360, a 234x reduction. That ratio is the explicit price of *not* committing to locality and translation equivariance.

4. Forward pass, in code

The `Conv2D` layer in `scratchnn` holds `kernels` of shape $(C_{\text{out}}, C_{\text{in}}, k, k)$ and `bias` of shape $(C_{\text{out}},)$. Inputs and outputs are flat `list[float]`, indexed as `c * H * W + r * W + col` for channel `c`, row `r`, column `col`. This keeps the rest of the library (`Tanh`, `ReLU`, `Linear`) working unchanged: from those layers' point of view, the conv output is just a flat vector of size $C_{\text{out}} \cdot H' \cdot W'$.

The forward pass is five nested loops with no NumPy:

```
for c_out in range(C_out):
    kernel = kernels[c_out]
    b = bias[c_out]
    for r in range(out_h):
        for col in range(out_w):
            z = b
            for c_in in range(C_in):
                for i in range(k):
                    for j in range(k):
                        z += kernel[c_in, i, j] * x[c_in, r + i, col + j]
            out[c_out, r, col] = z
```

Five loops looks unflattering. It is also exactly as expensive as the math suggests: $C_{\text{out}} \cdot C_{\text{in}} \cdot k^2$ multiply-adds per output spatial position, times $H' \cdot W'$ positions. For our 8x8 digits with the architecture above: $4 \cdot 1 \cdot 9 \cdot 36 = 1296$ multiply-adds per forward pass. The five-nested-loop structure makes the expense visible in the code, which is exactly the pedagogical point.

The input `x` is cached for the backward pass, the same pattern as `Linear`.

5. Backward pass, by careful chain rule

Let $g_{c_{\text{out}},r,c} = \partial L / \partial z_{c_{\text{out}},r,c}$ be the gradient flowing into the output feature map. We need three quantities: gradients with respect to the kernel weights, with respect to the bias, and with respect to the input.

Kernel gradient. Each weight $W_{c_{\text{out}},c_{\text{in}},i,j}$ participates in the output at *every* spatial position (r, c). By the chain rule, its gradient sums over those positions:

$$\frac{\partial L}{\partial W_{c_{\text{out}},c_{\text{in}},i,j}} = \sum_{r,c} g_{c_{\text{out}},r,c}\, x_{c_{\text{in}},\, r+i,\, c+j}.$$

This is the familiar `Linear` formula $g_i\, x_j$, summed over the spatial axes. The reuse of W in the forward pass is *exactly* what causes the gradient to accumulate in the backward pass.

Bias gradient. The bias adds to every output position equally, so its gradient is the sum of incoming gradients on its feature map:

$$\frac{\partial L}{\partial b_{c_{\text{out}}}} = \sum_{r,c} g_{c_{\text{out}},r,c}.$$

Input gradient. The input pixel $x_{c_{\text{in}},r',c'}$ was read by output positions (r, c) such that $r + i = r'$ and $c + j = c'$, that is, $(r, c) = (r' - i, c' - j)$. Summing over all kernels and the valid positions where the pixel was used:

$$\frac{\partial L}{\partial x_{c_{\text{in}},r',c'}} = \sum_{c_{\text{out}}} \sum_{i,j} W_{c_{\text{out}},c_{\text{in}},i,j}\, g_{c_{\text{out}},\, r'-i,\, c'-j},$$

with terms whose indices fall outside the valid range dropped. This is the spatial transpose of the forward pass; in convolutional-network jargon it is called *transposed convolution*.

The crucial observation: every accumulation in `Conv2D` is the *same* `+=` pattern the library already uses for mini-batch gradients in `Linear`. The mini-batch sums per-example gradients; the conv sums per-position gradients. Same operator, finer grain. The foundations post's closing note on autograd (per-layer becomes per-operation) is one more step along the same axis.

Numerical gradient check (run `tests/test_gradients.py` after adding a `gradient_check` case for `Conv2D`) confirms the math: worst relative error is on the order of 10^{-9} across small random inputs, well under the standard 10^{-4} tolerance.

6. Training on 8x8 digits

We use the same dataset and training loop as the foundations post's MLP: UCI optdigits (3823 training, 1797 test, 10 classes), pixel values normalized to $[0, 1]$, mini-batch SGD. Hyperparameters: 40 epochs, learning rate 0.1, batch size 32. The MLP run shown in the table below uses the same settings, so the comparison is apples-to-apples. Both models train in a few minutes of pure Python. The full demonstration is `examples/digits_cnn.py`.

Model	Architecture	Parameters	Test accuracy
MLP	$64 \to 32 \to 10$	2410	96.1%
CNN	`Conv2D(1, 4, k=3)` $\to$ ReLU $\to$ `Linear(144, 10)`	1490	95.4%

The CNN is about 38% smaller in parameter count and lands within a percentage point of the MLP. It does not dominate. Two possible readings:

1. The CNN's prior is wrong for this data (locality does not help).
2. The CNN's prior is correct, but the data has already had locality *baked into the preprocessing*, so the CNN does not gain much from asserting it again.

Reading (2) is the right one. The 8x8 UCI digits are not raw images. They were produced from the original 32x32 NIST bitmaps by counting "on" pixels in 4x4 blocks. That 4x4 block-counting step is itself a fixed translation-invariant local feature extractor: each 8x8 pixel is already the result of a small local aggregation over a contiguous 4x4 patch. By the time the CNN gets the data, the

spatial work is largely done. On raw 28x28 MNIST or actual photographs, the gap between MLP and CNN is much larger (a few percentage points to tens), precisely because the spatial structure has not been pre-summarized away.

To turn the question "is the CNN actually using its locality prior?" into an experiment, we run the **permuted-pixel control**.

7. The permuted-pixel control

Apply a single fixed random permutation to the 64 input pixels. Train and test on the permuted version. For a human, the resulting "images" are unreadable: digits are completely scrambled. For the MLP, every pixel position is interchangeable anyway, so training accuracy should be essentially unchanged. For the CNN, the locality assumption is now *actively wrong*: neighbors in the permuted input are no longer neighbors in the original digit, so a 3x3 kernel sees a meaningless 3x3 window of unrelated pixels.

Same training regime as above, with the same fixed permutation applied to both models:

Model	Pixels	Test accuracy	Drop from standard
MLP (64-32-10)	standard	96.1%	
MLP (64-32-10)	permuted	95.7%	−0.45 pp
CNN (1, 4, k=3)	standard	95.4%	
CNN (1, 4, k=3)	permuted	93.9%	−1.51 pp

The MLP drops by about 0.4 percentage points. The CNN drops by about 1.5, more than *three times* as much. That gap is the falsification test. If the CNN were ignoring its locality prior (treating the restricted receptive field as a bottleneck without using it as a useful constraint), it should drop by the same tiny amount as the MLP. It does not. The CNN is genuinely using spatial adjacency. We can tell because removing it costs the CNN more.

This is the experimental signature of an inductive bias in operation. The *direction* of the gap (CNN worse off under permutation) confirms the prior is doing real work. The *magnitude* of the gap (small in absolute terms, because this dataset has been pre-pooled) reflects how much spatial structure survives into the preprocessed features.

The two findings together say something the headline "CNN matches MLP with fewer parameters" alone would not: the CNN's parameter efficiency is not an accident or an artifact of the head being thin. It comes from the conv kernel actually exploiting adjacency. When adjacency is removed, the efficiency is gone.

8. Inductive bias, named

The CNN commits to two priors. Made explicit:

- **Locality.** A unit reads a $k \times k$ window, not the whole image. Long-range interactions only emerge by stacking conv layers (each output cell's "receptive field" grows with depth).
- **Translation equivariance.** The same weights apply at every position. Shift the input, shift the output.

What is given up:

- **Free interaction between distant pixels.** An MLP can wire the upper-left corner directly to the lower-right with a single weight. A single conv layer cannot represent that at all; deeper conv stacks can, but only indirectly, through receptive-field growth over depth.
- **Position-specific filters.** If the digit's location in the image carries information, the CNN has to learn to compensate elsewhere, or be given extra channels, or be helped by external normalization (such as centering the digit before input).

This is the trade. An MLP is the most generic supervised model: every input may interact with every other on equal footing. A CNN says: most useful interactions on images are local and translation-equivariant. That belief, baked into the weights through structural reuse, is what lets the CNN match the MLP with fewer parameters here, and outperform it (often dramatically) on data where the spatial structure has not been pre-aggregated.

The foundations post named two axes of inductive bias: output head and architecture. This post covered the architecture axis on its first non-trivial commitment, which is to a specific group of symmetries (2-D translations). The next post in the series makes the same kind of move on a different data type: sequences (text, time series) have their own notion of locality (recent past matters more than distant past) and their own translation symmetry (time-shift instead of space-shift). A fixed-context language model and, later, a recurrent network commit to those priors by reusing weights *across time steps* rather than *across spatial positions*. Different data, same lesson: structural weight sharing is how a network commits to an invariance prior, and committing to the right prior is how it learns more from less.

9. What stayed pure Python, and what will not

The five-nested-loop `Conv2D` worked here because the input is tiny (8x8, single channel), the kernels are small ($k = 3$), and the channel count is modest (4 output channels). The forward pass is about 1300 multiply-adds per example; the backward pass is roughly the same. For 3823 training examples and 40 epochs, training finishes in a few minutes of pure Python.

This is the largest layer we will write in pure Python. For 28x28 MNIST with three conv layers and tens of channels each, the inner loop count explodes by two orders of magnitude. At that point the five nested loops are not unflattering, they are *intractable*. A vectorized implementation, first in NumPy and eventually with batched tensor operations in a framework like PyTorch, becomes genuine relief rather than premature optimization. The math has not changed, the indices have not changed, only the implementation switches from interpreted Python loops to compiled tensor operations.

That switch is the topic of the Transformer post later in the series. By then, pure Python is comprehensively out of budget, and we will lean on NumPy without apology because the math has been derived by hand and vectorization is the optimization.